
Real-World PBTs as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

We present a benchmark for evaluating AI models and agents on real-world formal software verification tasks. We first scrape $\approx 20k$ property-based tests (PBTs) from permissively licensed real-world Python repositories, then automatically translate them into Lean 4 specifications with sorry placeholders. Translating PBTs into Lean specifications is challenging: it requires modeling Python semantics in Lean, inferring the logical property encoded in an imperative PBT, and handling the inherent difficulties of dependently-typed programming in a seldom-used language. We describe a three-agent LLM pipeline for transpiling PBTs into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof generation using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software, which we claim is of increasing importance as AI produces more and more of the world’s code.

1 Introduction

Several AI safety proposals are premised on implementing safeguards by way of *formal verification* (FV)¹ – that is, software systems that can produce and unequivocally certify mathematical proofs. These systems can be split into *interactive theorem provers* (ITPs) which accept and validate user-supplied proofs, and *automated theorem provers* (ATPs) which attempt (but often fail) to prove theorems fully autonomously. ARIA Safeguarded AI [8], Guaranteed Safe AI [7], and Scalable Formal Oversight [41, 30], among others, all suggest that in the future, all AI-generated software will be formally verified by autonomous FV agents, who will steer ITPs in much the same way that coding agents currently steer software production.

The only way FV can play this role is if agents can conduct verification work at least as well and as quickly as they can produce code. Otherwise, FV becomes a bottleneck and will naturally wither under the capital pressure to iterate². There is currently little evidence that such capabilities are possible. To achieve them, we would first need high-quality benchmarks to train on. Existing benchmarks (see Table 1) are too small [26], focused on advanced mathematics rather than engineering [14], or derived from expert-driven verification projects [6] which are quite different from what you would get were you to verify the average software project. To clarify this last point: historically, FV has been bottle-necked by the number of human expert practitioners able to write proofs in an ITP³, and therefore, FV was only ever applied to the most mathematically complex and safety-critical

¹FV forms a subset of *formal methods*, which also include PBT, model checking, and abstract interpretation.

²The reader is referred to [34] for a detailed discussion of how socioeconomic pressures degrade safety cultures and cause predictable accidents.

³We were unable to find any formal estimate of the number of human formal methods practitioners today. However, as a very rough proxy, de Almeida Borges et al. [9] conducted the largest ever survey of users of any particular ITP – in this case Coq, which is now called Rocq – at 466 participants. As a point of contrast, that year’s survey by the Rust foundation of Rust software engineers had 7,156 participants [37]. Anecdotally, most ITP work today is in Rocq, Lean 4, or Isabelle.

applications, such as compilers [23] or an OS kernel [21]. These projects are generally monolithic, closed systems, in contrast to the average software project which may have a frontend and backend, a database, reliance on several external black-box APIs such as an inference provider or a logging service, and an unnecessarily large dependency tree. The latter category of (normal) software is actually in many ways much more challenging to verify because it necessitates reasoning about black-box external systems and the interactions of several different components, potentially written in different languages and operating on different chips [4, 32].

MvH: A reviewer is going to ask us how we handle all this complexity. Do we have theorems that relate to external APIs? In what ways do we REALLY differ from other benchmarks? We need to substantiate this.

Benchmark	Lean	Isabelle	Coq/Rocq	HOL Light	Metamath	Dafny	Verus	Theorems	Focus
Minif2f [45]	✓	✓		✓	✓			488	math Olympiad
PutnamBench [40]	✓	✓	✓					640	math Olympiad
Fimo [25]	✓							149	math Olympiad
DafnyBench [26]						✓		782	scraped Dafny proofs
ProofNet [1]	✓							371	U math
RLM25 [33]	✓							619	G math
Lean Workbook [43]	✓							57,000	HS, U, G math
FormalMATH [44]	✓							5,560	math Olympiad
LEAN-GitHub [42]	✓							28,597	scraped Lean proofs
ProverBench [35]	✓							325	math Olympiad
MiniCTX [17]	✓							762	six Lean projects
LeanGeo [36]	✓							122	HS, U, G math
IneqMath [28]	✓							1,522	math Olympiad
FATE- $\{M,H,X\}$ [20]	✓							350	U, G math
VeriBench [3]	✓							857	software verification
FVAPPS [13]	✓							4,715	software verification (from APPS)
CLEVER [38]	✓							161	software synthesis & verification
Vericoding [5]	✓					✓	✓	12,504	software synthesis & verification
MUSTARD [18]	✓							5,866	HS, U math
FormL4 [27]	✓							14,510	U math
CoqStoq (Rango) [39]			✓					10,396	software verification
MSC-180 [24]	✓							180	U, G math
HOList [2]				✓				29,462	U, G math

Table 1: **Prior ITP benchmarks.** HS = high-school, U = undergraduate, G = graduate. If one benchmark improves upon another, we exclude the lesser. 3 indicates the proof assistants each benchmark targets.

In light of the limitations of prior works, we generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a “close sibling” technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams [12]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering, and provides only statistical assurance.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s [11], any unproven theorem statement is automatically treated as a PBT until the author supplies a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard.

We build on this. First, we scrape $\approx 20k$ permissively-licensed PBTs written in Python’s Hypothesis framework [29]. Each PBT in our dataset can be seen as a formal conjecture about a piece of Python code, which the author would like to either substantiate (prove) or disprove. To make it possible for an AI to resolve these conjectures, we translate both code and PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. Once the Lean version of the conjecture is (eventually) proven, it can be called a *theorem*. Until then, it has a `sorry` placeholder where the proof belongs. This approach was pioneered by [13] on the FVAPPS verification benchmark. In FVAPPS, source programs were taken from the APPS benchmark by [16], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI agents to translate from Python to Lean, in the style of FVAPPS. This process is necessarily lossy, that is to say, it does not guarantee strict equivalence. However, strict equivalence is only needed if the goal is to aid the authors of the original software by resolving their open conjectures. Although when applicable we consider this goal a positive secondary contribution of our work, our actual motivation is to build a large benchmark of realistic and high-quality software verification exercises, representative of the diversity of code found in “normal” codebases. Thus, we don’t worry whether or not our translations are perfectly equivalent. The result is tens of thousands of verification challenges. Each consists of four artifacts (Figure 1): (1) the original Python implementation, (2) the original Python PBT exercising it, (3) a complete Lean implementation of the same function (no `sorry`), and (4) a Lean theorem statement of the property with a `sorry` for the proof. Unlike existing FV benchmarks, each problem corresponds to real-world software written by engineers with no formal verification experience: as Table 1 shows, prior software-verification benchmarks are scarce (the bulk of ITP benchmarks target mathematics) and small, and none derive from code that was authored outside of a theorem-proving context, putting our problems out of distribution relative to anything an AI is likely to have memorised.

MvH: TODO: Summary of results.

The rest of the paper is organized as follows. We formalize the task in Section 2. We characterize our input corpus, detailing how we scraped and deduplicated $\approx 20k$ permissively-licensed Hypothesis tests from open-source repositories, in Section 3, and discuss how that corpus compares to a similar one released by the Hypothesis team [10]. Then, we describe our generation pipeline—scraping, dependency extraction, LLM-based transpilation with iterative Lean LSP repair, and post-production cleaning—in Section ???. We present baseline results in Section ??, discuss threats to validity in Section ??, survey related work in Section ??, and conclude in Section ??.

2 Task Definition

Each problem in the FVSpec benchmark consists of four components: (1) the original Python implementation of the function under test, (2) the original Python PBT exercising that implementation, (3) a Lean implementation of the same function—a fully defined, computable definition, no `sorry`—and (4) a Lean theorem corresponding to the PBT, with a `sorry` placeholder where the proof should go. The AI’s task is to fill in every theorem `sorry` with a Lean proof term that compiles cleanly under `lake build`; submissions are then scored by whether the proof retains any `sorry`, `admit`, or axioms beyond Mathlib’s defaults. The Lean kernel is the sole arbiter—we perform no separate semantic comparison against the original Python. Each problem also includes a URL pointing to the original source code and license information.

As a running example, consider `isort(1st)`. Figure 1 shows all four artifacts of a single FVSpec problem. The left column holds the original implementation (top) and the PBT exercising it (bottom); the right column holds the corresponding `Impl.lean` (top), a complete Lean port of the function, and `Spec.lean` (bottom), containing two theorems—one for sortedness, one for permutation—each with their own `sorry`. The AI’s task is to discharge both Lean `sorry` placeholders.

We encounter two practical issues when translating Python PBTs into Lean implementations and (unproven) specifications.

Nonsensical theorems. Our pipeline can emit well-formed but useless statements in three ways.

(i) *Trivial Unit-typed conjectures.* When the formalization agent cannot recover a function’s true return type, it falls back on `Unit` as a placeholder. The resulting theorem is a tautology—e.g. for

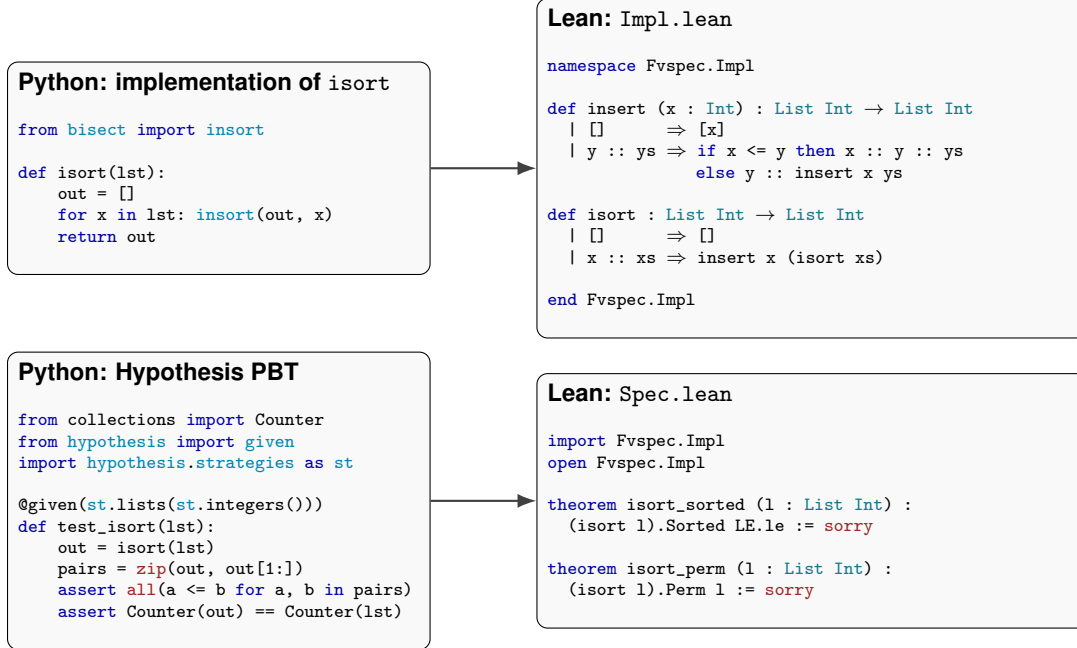


Figure 1: The four artifacts of a single FVSpec problem. Top: the Python implementation of `isort` and its complete Lean port. Bottom: the Hypothesis PBT and the corresponding Lean theorems. The AI must replace both Lean `sorry` placeholders—one per theorem.

107 our running example, `theorem isort_check : isort lst = () := by plausible`, which
 108 holds for any total function into `Unit` and so carries no verification value.

109 (ii) *Ill-typed or unresolved-symbol theorems.* The formalization agent works from the Python
 110 source and writes Lean by analogy. When it invents a Lean name that does not exist—for in-
 111 stance, writing `theorem isort_perm (l : List Int) : (isort l).permutationOf l`
 112 `:= sorry`, where `permutationOf` is not a member of `List` (the correct name is `List.Perm`)—
 113 or attaches a type signature inconsistent with the surrounding code, the Lean elaborator rejects the
 114 file with errors like *unknown identifier*, *type mismatch*, *function expected*, or *failed to synthesize*.

115 (iii) *Spec theorems leaking into Impl.lean.* Our pipeline runs two separate agents: one writes
 116 `Impl.lean` (definitions only), the other writes `Spec.lean` (theorems only). The implementa-
 117 tion agent is explicitly instructed to emit no theorems, but occasionally—when shown the `isort`
 118 source—it pre-empts the spec agent by hallucinating a `namespace Fvspec.Spec ... theorem`
 119 `isort_sorted ... end Fvspec.Spec` block inside `Impl.lean`, which violates our file split and
 120 would silently expand the proof obligation if left in.

121 We catch (i) by regex, drop (ii) at compile time, and strip (iii) with a namespace-aware filter. A
 122 theorem that survives all three filters is a useful challenge problem even if it has drifted from the
 123 original PBT—because Lean checks proofs against *the theorem we generated*, not the informal
 124 property.

125 **Lean as a (nearly) perfect oracle.** The Lean kernel arbitrates correctness, which is qualitatively
 126 much stronger than statistical sampling. It is not, however, adversarially perfect: an AI can satisfy the
 127 kernel by *changing the question*—adding the Axiom of Choice, weakening hypotheses, or exploiting
 128 a security vulnerability in Lean itself [15]—and the proof will still type-check [31]. To make gaming
 129 visible, every submission is scored on two axes: a binary *proved* flag (the file compiles under `lake`
 130 `build` and every original `sorry` has been discharged) and partial credit (fraction of original `sorry`s
 131 discharged). We additionally record any axiom dependencies the proof introduces, since an AI can
 132 trivialize a goal by quietly importing strong axioms; we do not currently auto-fail on extra axioms,
 133 but report them so reviewers can flag suspicious solutions. For higher-stakes evaluations, submissions
 134 can additionally be re-checked through `leanprover/comparator` [22], which sandboxes the build

135 and cross-validates the proof term against an independently-implemented Rust kernel. Lastly, a
136 sufficiently motivated AI can still smuggle a vacuous proof past us when the original theorem
137 statement is itself too weak—a limit shared by every benchmark whose scale exceeds full human
138 review capacity.

139 Next, we explain how we scrape the PBT corpus and what the corpus looks like. **MvH: Nothing**
140 **below this has been revised yet ...**

141 3 Dataset Characterization

142 3.1 Input corpus: scraping and deduplication

143 **Scraping.** Source PBTs come from our RealPBT dataset (citation omitted for double-blind re-
144 view)⁴, assembled by a custom Python scraper that operates on the GitHub dependency graph of
145 the Hypothesis library. The scraper first built the candidate repository list by crawling the GitHub
146 dependency graph of the Hypothesis library [19] page by page using the PyGitHub API, collecting
147 19,808 unique owner/repository pairs into a static snapshot.⁵ For each candidate, the scraper iterates
148 asynchronously with rate-limit-aware GitHub REST API backoff, shallow-clones the repository, and
149 uses ripgrep to locate files containing the `@given` decorator. For each such file, it parses the Python
150 AST to extract all PBT functions—identified as function definitions whose decorator list includes a
151 call to `given`—and performs a transitive static call-graph traversal to recover the minimal closure of
152 locally-defined helpers called by each PBT. Repository metadata (name, URL, license, stars, forks,
153 commit hash) and extracted PBTs (source code, dependency names and their source) are persisted
154 to PostgreSQL; repos whose commit hash was already processed are skipped, enabling incremental
155 re-scraping. Only repositories whose license matches a permissive allowlist (MIT, Apache-2.0, BSD
156 variants, ISC, Unlicense, CC0, BlueOak-1.0.0) contribute PBTs to the released dataset.

157 **Fork deduplication.** The raw scrape, run with a 24-hour time budget, yielded 54,345 PBTs, but
158 GitHub’s dependency graph surfaces many forks of the same upstream (e.g., hundreds of forks of
159 `pytorch` each carrying an identical Hypothesis test suite). We deduplicate by grouping repositories
160 whose names share the same base component, selecting a canonical representative per group (highest
161 star count, breaking ties by PBT count), and retaining PBTs from canonical repos plus any PBTs
162 unique to forks (identified by SHA-256 of whitespace-normalized code). After fork deduplication the
163 corpus contains 11,039 PBTs across 333 repositories from 281 unique GitHub users.

164 3.2 Corpus statistics

165 Table 2 summarizes the post-deduplication input corpus; the full post-production pipeline that
166 produces the final benchmark from these PBTs is described in Section ???. The star and fork
167 distributions are heavily right-skewed: the median repository has 0 stars, reflecting that most real-
168 world Hypothesis users work on small or personal projects. The median PBT is 13 lines with a
169 cyclomatic complexity of 2, consistent with the concise, assertion-focused style typical of Hypothesis
170 tests. Halstead effort, a measure of the cognitive work required to write the code, has a median of 3
171 and a long right tail (mean 207, 95th percentile 981), consistent with a corpus dominated by short,
172 simple tests alongside a minority of complex multi-strategy properties.

173 Figure 2 characterizes the relationship between PBT complexity and the complexity of the code each
174 PBT tests. On average, PBTs are concise (median 18 LOC) while their direct local dependencies
175 total roughly 30× more code (mean 553 dep LOC). The ratio is right-skewed: many PBTs exercise
176 a handful of short helper functions, but a substantial tail tests deeply nested call graphs spanning
177 hundreds of lines. Notably, PBT complexity is nearly independent of dependency complexity (Pearson
178 $r = 0.06$ between test cyclomatic complexity and dependency count), confirming that Hypothesis
179 encourages engineers to write simple, focused tests regardless of how complex the underlying code
180 is.

⁴The dataset will be released on HuggingFace upon de-anonymization.

⁵The dependency-graph crawl was run once offline; the resulting `dependents.json` snapshot is read by the main scraper at startup.

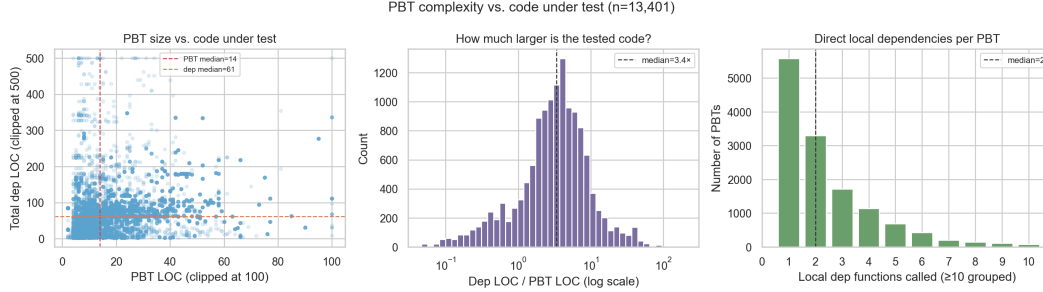


Figure 2: PBT complexity versus the complexity of the code each PBT tests, measured across 26,642 PBTs with recoverable local dependencies. *Left*: scatter of PBT LOC against total local-dependency LOC. *Centre*: distribution of the dep/PBT LOC ratio (log scale). *Right*: distribution of local project functions called directly per PBT.

Comparison with the Hypothesis Corpus. DeVoe [10] recently released the Hypothesis Corpus 2026 (HC-2026): 28,928 Hypothesis tests from 1,504 repositories, assembled by the Hypothesis development team to study PBT *runtime behaviour*—how generators consume entropy, how tests are distributed in time, which predicates fire. We built RealPBT for a different purpose: supplying clean, license-clear, dependency-annotated PBTs for Lean formalization. These divergent goals drive the key design differences summarised in Table 3: HC-2026 is richer in runtime context (timing, coverage, entropy), while RealPBT provides what formal verification requires (dependency source, static metrics, NL summaries) with a stricter license filter. Table 3 also shows the repository overlap we computed programmatically: 28 exact matches after their validity filter, 111 (33%) before it. A full methodological comparison is given in Section ??.

Table 2: Summary statistics for the post-deduplication RealPBT input corpus.

Metric	Value
PBTs (pre-dedup, HuggingFace)	54,345
PBTs (post-dedup, working set)	11,039
Unique repositories	333
Unique GitHub users	281
Median PBTs per repo	6
Median repo stars	0
Mean repo stars	39
Median PBT LOC	13
Median cyclomatic complexity	2.0
Median Halstead effort	3
MIT-licensed repos	148
Apache-2.0-licensed repos	66
BSD-3-Clause-licensed repos	25

Table 3: Comparison of the FVSpec RealPBT corpus with the Hypothesis Corpus 2026 [10]. Both datasets deduplicate repos; only ours deduplicates at the test level.

	Ours	HC-2026
Collection	dep. graph	API search
# PBTs (pre-dedup)	54,345	~28,928 ^a
# PBTs (post-dedup)	11,039	23,139
# Repos	333	1,504
Repo dedup	fork/name	MinHash
Test dedup	SHA-256	none
License filter	perm. only	none
Dep. source code	✓	×
Code metrics	✓	×
NL summary	✓	✓
@settings	×	✓
Runtime data	×	✓
Coverage	×	✓
Repo overlap (valid)	28 / 333 (8%)	
Repo overlap (all)	111 / 333 (33%)	

^a 28,928 canonical (parametrization-deduped) nodes per their schema; 23,139 remain after code-level deduplication.

References

- [1] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

- [2] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine learning of higher order logic theorem proving. In *International conference on machine learning*, pages 454–463. PMLR, 2019.
- [3] S. Barkallah, S. Daruru, B. Miranda, L. Aniva, A. Nie, and S. Koyejo. VeriBench-FTP: A formal theorem proving benchmark in Lean 4 for code verification. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [4] S. Buro, R. Crole, and I. Mastroeni. Equational logic and categorical semantics for multi-languages. *Electronic Notes in Theoretical Computer Science*, 352:79–103, 2020.
- [5] S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [6] S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [7] D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- [8] N. A. David Dalrymple. Safeguarded ai. <https://aria.org.uk/opportunity-spaces/mathematics-for-safe-ai/safeguarded-ai/>, 2025. Accessed 25 April 2026.
- [9] A. de Almeida Borges, A. Casanueva Artís, J.-R. Falleri, E. J. Gallego Arias, É. Martin-Dorel, K. Palmiskog, A. Serebrenik, and T. Zimmermann. Lessons for interactive theorem proving researchers from a survey of coq users. *Journal of Automated Reasoning*, 69(1):8, 2025.
- [10] L. DeVoe. The Hypothesis corpus. <https://hypothesis.works/articles/hypothesis-corpus/>, 2026. Accessed April 2026.
- [11] P. C. Dillinger, P. Manolios, D. Vroon, and J. S. Moore. Acl2s:“the acl2 sedan”. *Electronic Notes in Theoretical Computer Science*, 174(2):3–18, 2007.
- [12] M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.
- [13] Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- [14] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.
- [15] K. Gopinathan. Lean proved this program was correct; then I found a bug. <https://kirancodes.me/posts/log-who-watches-the-watchers.html>, Apr. 2026. Blog post; reports a heap buffer overflow in `lean_alloc_sarray` in the Lean 4 runtime, found by AFL++ fuzzing of the verified `lean-zip` library. Accessed 26 April 2026.
- [16] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [17] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024.

- [18] Y. Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.
- [19] HypothesisWorks. Dependents of Hypothesis on GitHub. <https://github.com/HypothesisWorks/hypothesis/network/dependents>, 2024. Accessed 2024.
- [20] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- [21] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [22] Lean FRO. Comparator: A trustworthy judge for Lean proofs. <https://github.com/leanprover/comparator>, 2026. Sandboxes the build and replays proof terms, optionally cross-checking against an independently-implemented Rust kernel (nanoda). Accessed 26 April 2026.
- [23] X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. Compcert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [24] S. Li, W. Lu, X. Shi, K. Weng, H. Sun, M. Yu, T. Zhang, G. Yu, H. Liu, and L. Du. MSC-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*, 2025.
- [25] C. Liu, J. Shen, H. Xin, Z. Liu, Y. Yuan, H. Wang, W. Ju, C. Zheng, Y. Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. *arXiv preprint arXiv:2309.04295*, 2023.
- [26] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [27] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.
- [28] P. Lu, J. Sheng, L. Lyu, J. Jin, T. Xia, A. Gu, and J. Zou. Solving inequality proofs with large language models. In *The 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [29] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.
- [30] E. Miyazono. Formally scalable AI oversight through specifications. <https://foresight.org/resource/evan-miyazono-formally-scalable-ai-oversight-through-specifications/>, 2024. Accessed 25 April 2026.
- [31] E. Miyazono. Lies, damned lies, and proofs: Formal methods are not slopless. <https://www.lesswrong.com/posts/rhAPh3YzhPoBNpgHg/lies-damned-lies-and-proofs-formal-methods-are-not-slopless>, Jan. 2026. Accessed 26 April 2026.
- [32] J. T. Perconti and A. Ahmed. Verifying an open compiler using multi-language semantics. In *European Symposium on Programming Languages and Systems*, pages 128–148. Springer, 2014.
- [33] A. Poiroux, G. Weiss, V. Kunčák, and A. Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17958–17980, 2025.

- 291 [34] J. Rasmussen. Risk management in a dynamic society: a modelling problem. *Safety science*, 27
292 (2-3):183–213, 1997.
- 293 [35] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al.
294 Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for
295 subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- 296 [36] C. Song, Z. Wang, F. Pu, H. Wang, X. Lin, J. Liu, J. Li, and Z. Liu. LeanGeo: Formalizing
297 computational geometry problems in Lean. *arXiv preprint arXiv:2508.14644v1*, 2025.
- 298 [37] R. S. Team. 2025 state of rust survey results. [https://blog.rust-lang.org/2026/03/](https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/)
299 [02/2025-State-Of-Rust-Survey-results/](https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/), March 2026. Accessed 25 April 2026.
- 300 [38] A. Thakur, J. Lee, G. Tsoukalas, M. Sistla, M. Zhao, S. Zetsche, G. Durrett, Y. Yue, and
301 S. Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv*
302 *preprint arXiv:2505.13938*, 2025.
- 303 [39] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira,
304 S. Lerner, and E. First. Rango: Adaptive retrieval-augmented proving for automated software
305 verification. *arXiv preprint arXiv:2412.14063*, 2024.
- 306 [40] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri.
307 Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition.
308 *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- 309 [41] M. von Hippel. [https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/](https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program)
310 [the-scalable-formal-oversight-research-program](https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program), 2026. Accessed 25 April
311 2026.
- 312 [42] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a
313 versatile Lean prover. *arXiv preprint arXiv:2407.17227*, 2024.
- 314 [43] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean
315 problem set formalized from natural language math problems. *Advances in Neural Information*
316 *Processing Systems*, 37:105848–105863, 2024.
- 317 [44] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al.
318 Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv*
319 *preprint arXiv:2505.02735*, 2025.
- 320 [45] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level
321 mathematics. *arXiv preprint arXiv:2109.00110*, 2021.